# A Web Application with Mongita

Pets, owners, and the database layer

Gregory S. DeLozier, PhD

# Follow One Pet

Create Casey with Alex.

Reassign Casey to Blair.

Try deleting Blair.

**Which records change at each step?**

# Three Parts, Three Jobs

| Part | Job |
| --- | --- |
| `app.py` | Requests and responses |
| `templates/` | Forms, links, displayed values |
| `database.py` | Validation and document operations |

**Routes ask for records. The layer retrieves them.**

# Start the Application

```
cd chapter-12-mongo-app/code
python -m pip install -r requirements.txt
flask --app app run --debug
```

Create an owner before creating a pet.

Local storage: **`code/data/mongita`**

# Two Collections, One Reference

| Collection | Document fields |
| --- | --- |
| `owners` | `_id`, name, city, type_of_home |
| `pets` | `_id`, name, type, age, owner_id |

**The pet's `owner_id` identifies an owner document.**

# A Form Carries the Owner’s ID

```
<select name="owner_id">
  {% for owner in owners %}
  <option value="{{ owner['id'] }}">
    {{ owner['name'] }}
  </option>
  {% endfor %}
</select>
```

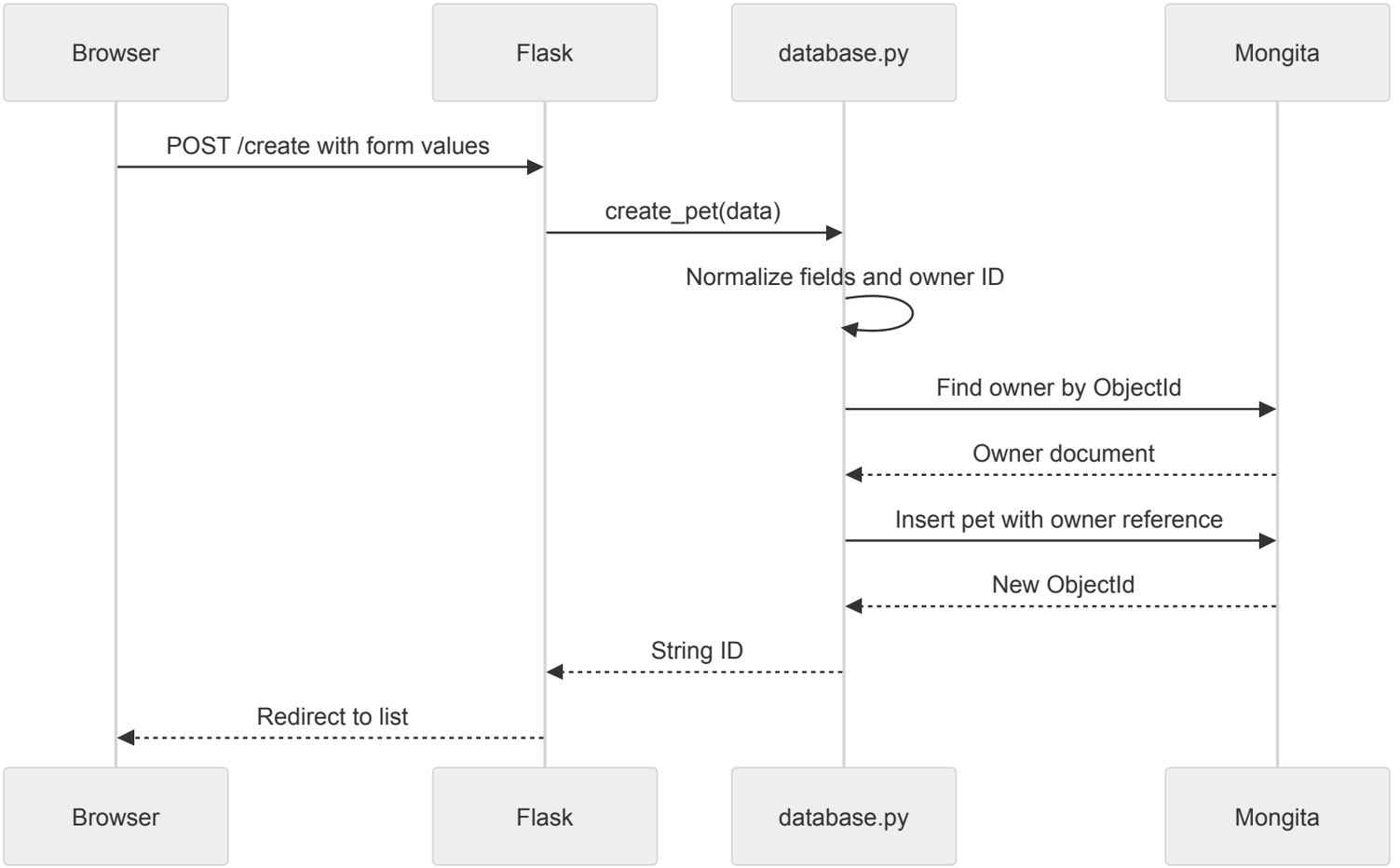
Visible name: **Alex**. Submitted value: **Alex’s ID**.

# IDs at the Boundary

| Place | Value |
| --- | --- |
| Saved `_id` and `owner_id` | `ObjectId` |
| Route and template dictionaries | String |
| Form or web address | String |
| Database lookup filter | Converted `ObjectId` |

## Convert inside `database.py`.

# Create: Follow the Save



# Check Before Inserting

```
def create_pet(data):
    pet = _normalize_pet_data(data)
    result = pets_collection.insert_one(pet)
    return str(result.inserted_id)
```

Required name and type. Existing owner.

**Blank age becomes zero; invalid age is rejected.**

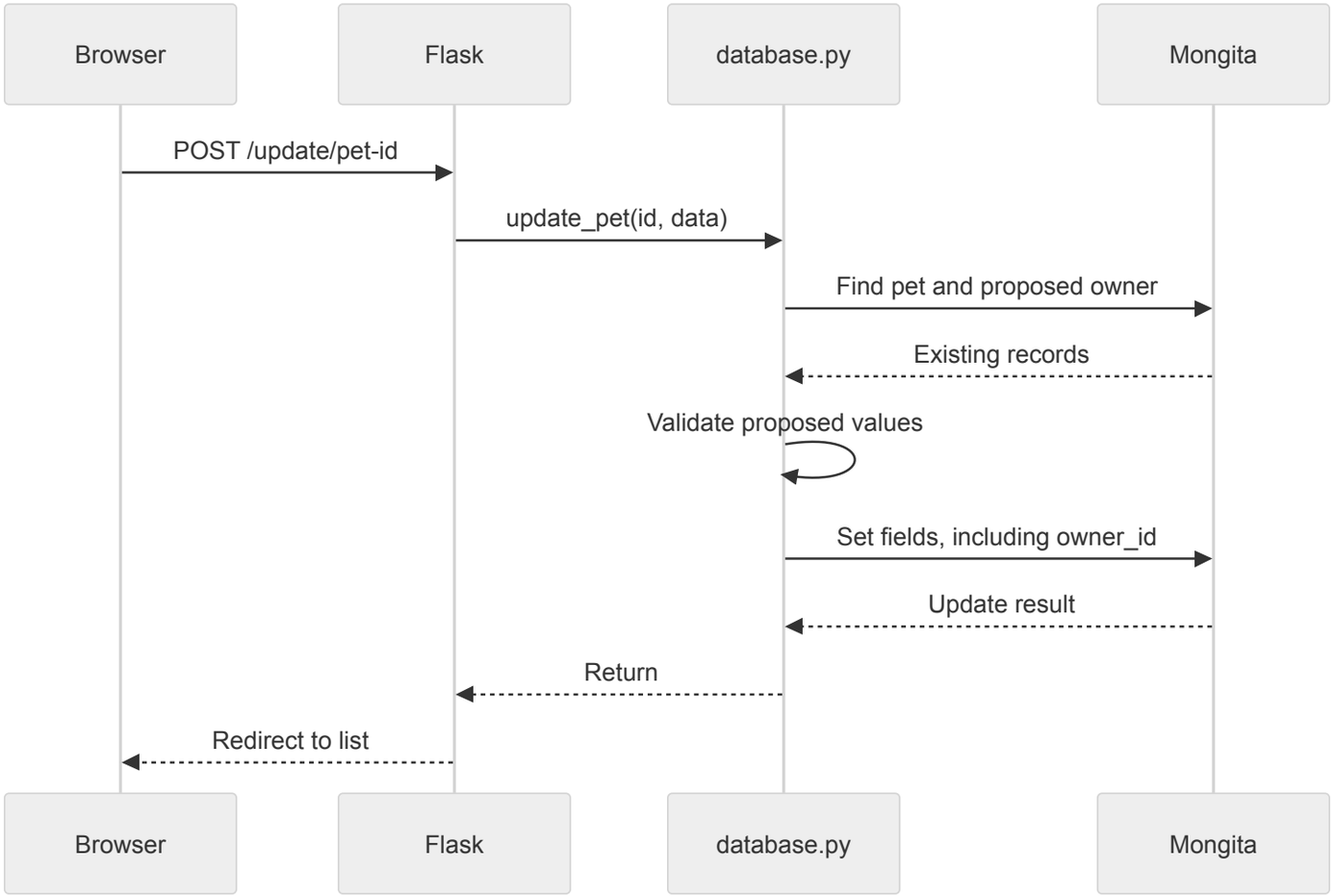
# Load the Update Form

`get_pet(id)` retrieves Casey.

`get_owners()` supplies the choices.

The form selects **Casey's current owner**.

# Update: Follow the Save



# Reassigning Casey

| Value | Before | After |
|---|---|---|
| Pet ID | Casey’s ID | Same ID |
| Age | 3 | 4 |
| `owner_id` | Alex’s ID | Blair’s ID |
| Owner records | Alex and Blair | Same two records |

**A changed reference, not a new pet.**

# A Rejected Update

Submit age **-5** for Casey.

Return to the list and reopen Update.

**Still age 4. Still owned by Blair.**

# Can We Delete Blair?

```
pet = pets_collection.find_one(
    {"owner_id": object_id}
)
if pet is not None:
    raise ConstraintError(...)
```

**One matching pet is enough to say no.**

# The Layer Enforces the Relationship

A pet must refer to an existing owner.

An owner with pets cannot be deleted through the layer.

**What must another writer preserve?**

# Different Failures, Different Responses

| Situation | Response |
| --- | --- |
| Invalid identifier text | 400 |
| Pet identifier finds no record | 404 |
| Owner still has pets | 400 |
| Successful save | 302 redirect |

**Valid identifier syntax does not prove existence.**

# Restart and Inspect

Stop Flask. Restart it. Read Casey again.

| Interface | Identifier type |
|---|---|
| `database.get_pet(id)` | `str` |
| Stored document | `ObjectId` |

**Python variables disappear. Saved documents remain.**

# Show the Owner’s Name

Add `owner_name` to the returned pet dictionary.

Keep the lookup inside `database.py`.

**How many owner lookups does the list need?**

# Keep the Application Contract Clear

The app passes **string identifiers**.

The layer checks values and related records.

Documents store the state needed for the next request.